COP 3502C Programming Assignment # 4

Cat Breed Rank Lists (Sorting)

Please read all the pages before starting to write your code

Please include the following commented lines at the beginning of your code to declare your authorship of the code:

/* COP 3502C PA4

This program is written by: Your Full Name */

Compliance with Rules: *The UCF Golden Rules apply to this assignment and its submission. In addition, all assignment policies outlined in the syllabus also apply here. The instructor or TA may ask any student to explain any part of their code to verify authorship and ensure clarity. Sharing this assignment description or your code, in whole or in part, with anyone or anywhere is a violation of course policy. Likewise, using code obtained from any outside source, including AI tools, will be considered cheating. Your primary resources should be class notes, lectures, labs, and TAs. These rules are in place to ensure fairness, maintain academic integrity, and give every student the opportunity to develop their own problem-solving and coding skills.*

Deadline:

Please check Webcourses for the official assignment deadline. Assignments submitted by email will not be graded, and such emails will not receive a response in accordance with course policy.

What to do if you need clarification on the problem?

If you need clarification on the problem, a discussion thread will be created on Webcourses, and you are strongly encouraged to post your questions there. You may also ask questions on Discord; if TAs are available, they can respond, and other students might also contribute, helping you get answers more quickly. For further clarification on the requirements, you are welcome to visit the TAs or the instructor during office hours.

How to get help if you're stuck?

As stated in the course policy, office hours are the primary place to seek help. While we may occasionally respond to emails, debugging requests are best addressed during office hours rather than through email.

QuickPaw vs MergePaw: The Purrfect Ranklist

Objective:

In this assignment, you will:

- Practice implementing Merge Sort and Quick Sort
- Practice sorting using multiple keys
- Practice working with structs and pointers
- Practice working with CompareTo function concept

Background story:

A local animal shelter has collected detailed information about all of its cats, including how cute, fluffy, agile, friendly, smart, and lazy each one is. Potential adopters often want to view the cats ranked by a specific trait to help them find their ideal pet. For example, one person might want to see the fluffiest cats first, while another person might be interested in the smartest or most friendly cats.

Your task in this assignment is to build the ranking system for the shelter. Given information about many cats and their trait scores, your program will sort the cats according to a selected trait and produce an official **Cat Ranklist**. The list will show the cats ordered from the highest score to the lowest score for the chosen trait.

Efficiency is important because the shelter may eventually store information about **thousands of cats**. To handle this efficiently, you will implement two different efficient sorting algorithms, merge sort and quicksort, as learned in class.

Required Structure:

```
typedef struct Cat {
    char *name;
    int scores[NUMTRAITS];
}Cat;
```

Solution Design

This assignment requires you to implement two separate programs that produce the same input and output behavior, but use different sorting algorithms internally.

You will write:

- One program that uses Merge Sort
- One program that uses Quick Sort

Both programs must read the input in the same format and produce identical output. The only difference between the two solutions is the sorting algorithm used.

For both versions of the program, when the size of a subarray becomes less than or equal to a threshold value of 30, the sorting method must switch to a quadratic-time sorting algorithm, insertion sort (An $O(n^2)$ algorithm covered in class). This hybrid approach improves efficiency because simple algorithms perform well on small arrays.

Each sorting algorithm must also be implemented using a wrapper function that initiates the sorting process. The wrapper function will call the actual recursive sorting function that performs the divide-and-conquer steps. The required prototypes for these functions are provided later in this document.

In addition, you must implement a comparison function that determines the ordering between two cats. This function will compare cats based on the selected trait and break ties alphabetically using the cat's name. Both sorting algorithms will use this comparison function when determining ordering.

Memory for the cat structures and their names must be dynamically allocated. The array of cat pointers must be dynamically allocated based on the value of n. Each cat name must also be dynamically allocated using exactly the amount of memory required for that string, and you must use the same strategy of dynamic memory allocation of strings that we have learned in class. Any other approach may result in -100 in the class.

Input:

The first line of input contains a single positive integer, n ($1 \le n \le 10^5$), representing the number of cats in the input. Each of the next n lines indicates information about a single cat. The format of each line is as follows:

```
name cuteScore fluffScore agileScore friendlyScore smartScore lazyScore
```

where:

- name is the name of the cat
  Each cat name is unique and consists of 1 to 12 lowercase letters.
- cuteScore is the cat's cuteness score
- fluffScore is the cat's fluffiness score
- agileScore is the cat's agility score
- friendlyScore is the cat's friendliness score
- smartScore is the cat's intelligence score
- lazyScore is the cat's laziness score

Each score will be an integer between *0 and 1000, inclusive.*

The final line of input contains an integer g ($0 \le g \le 6$), representing the trait for which you want the rank list. The values of **g** correspond to the following traits:

g	**Trait**
0	Cuteness
1	Fluffiness
2	Agility
3	Friendliness
4	Intelligence
5	Laziness
6	Total score across all traits

Output:

The first line of output will have the format: `Trait Ranklist`, where `Trait` is one of the following constant strings stored in this array that will be required in your code:

```
const char TRAITS[NUMTRAITS][MAXSIZE+1] = {"Cuteness", "Fluffiness",
"Agility", "Friendliness", "Intelligence", "Laziness", "Total"};
```

The index into this array corresponds to the input value g.

After this header line, print n lines representing the ranked list of cats. The cats must be sorted from highest score to lowest score according to the selected trait.

If two cats have the same score for the selected trait, the tie must be broken using the cat name in alphabetical order.

Each line must have the format: `x. name score`

where:

- x is the 1-based rank
- name is the cat's name
- score is the score for the selected trait (or total)

Required Printing Format

The cat's name must be printed with a field width of 15 characters, left-justified.

Use the following statement:

```
printf("%d. %-15s %d\n", rank, name, score);
```

Sample Input	Sample Output
5 whiskers 90 80 70 95 85 60 mittens 85 95 60 80 75 70 shadow 70 60 95 85 90 50 luna 95 85 75 90 80 65 oreo 60 70 80 60 65 95 0	Cuteness Ranklist 1. luna 95 2. whiskers 90 3. mittens 85 4. shadow 70 5. oreo 60

Implementation Requirements/Run Time Requirements

Please use the following constants, struct and constant array:

```
#define MAXSIZE 12
#define NUMTRAITS 7
#define TOTAL 6
#define BASECASESIZE 30

const char TRAITS[NUMTRAITS][MAXSIZE+1] =
{"Cuteness", "Fluffiness", "Agility",
 "Friendliness", "Intelligence",
 "Laziness", "Total"};
```

```
typedef struct Cat {
    char* name;
    int scores[NUMTRAITS];
} Cat;
```

Constant Explanation:

- MAXSIZE is the maximum size of all strings in the problem, including both cat names and trait names.
- NUMTRAITS is the total number of possible keys for sorting (six traits plus one for the total score).
- TOTAL is the index into the scores array indicating the total score for a cat.
- BASECASESIZE is the size at which the recursive sorting algorithm must switch to insertion sort. Although Insertion sort is $O(n^2)$, it is a very good choice for small arrays. .

Sorting Requirements

1. Your program must store the cats in an **array of pointers to Cat structures** (Cat **list). Each element in the array points to one dynamically allocated Cat.
2. You must implement the following compareTo function:

```
// Returns a negative integer if the cat pointed to by ptrC1
// "comes before" the cat pointed to by ptrC2 on the ranklist
// sorted by the trait indicated by the integer key, breaking ties
// by the cat's name in alphabetical order. Returns 0 if the
// two cats are identical and returns a positive integer if the
// cat pointed to by ptrC2 comes before the cat pointed to by ptrC1.

int compareTo(Cat* ptrC1, Cat* ptrC2, int key);
```

3. For the Merge Sort version of the program, you must implement the following wrapper function. This function starts the sorting process.:

```
   // Merge Sorts the array list of size n according to the trait
   // indicated by the integer key.
void mergeSort(Cat** list, int n, int key);
```

4. For the Merge Sort version of your program, your recursive function must have the following prototype:

```
// Performs a Merge Sort on list[low..high] according to the trait
// indicated by the integer key.
void mergeSortRec(Cat** list, int low, int high, int key);
```

5. For the Quick Sort version of your program, your wrapper function must have the following prototype:

```
// Quick Sorts the array list of size n according to the trait
// indicated by the integer key.
void quickSort(Cat** list, int n, int key);
```

6. For the Quick Sort version of your program, your recursive function must have the following prototype:

```
// Performs a Quick Sort on list[low..high] according to the trait
// indicated by the integer key.
void quickSortRec(Cat** list, int low, int high, int key);
```

7. For your Quick Sort implementation, you must select your pivot randomly for each partition step. This means you can choose any element within the current subarray at random to act as the pivot, which helps reduce the chance of encountering the worst-case scenario on already sorted or nearly sorted data.

8. All the sorting algorithms must be implemented based on the lecture. Using any approach not taught in the class will be counted as cheating and may result in penalty.
9. Do not use any global variables (if not specified).
10. You must free all dynamically allocated memory. Your program must have zero memory leaks to receive full credit.
11. Use standard input/output (scanf/printf). File I/O is not allowed.
12. For any dynamically allocated strings, you must use the approach discussed in the class. Any other method will result in a -100 penalty.
13. Your code must compile and run successfully on CodeGrade to receive any credit.
14. Submit well-structured and well-commented code to avoid penalties.

Deliverables

1. Please submit a source file `ranklist_ms.c`, via Webcourses->codegrade, for your solution to the problem that uses Merge Sort.

2. Please submit a source file `ranklist_qs.c`, via Webcourses->codegrade, for your solution to the problem that uses Quick Sort.